

TORCS AI Racing Competition

Windows Setup Guide

Native installation — no containers required

This guide walks you through setting up the full TORCS AI competition environment directly on Windows. Follow the steps in order.

Step	What you will do
1. Prerequisites	Hardware requirements, disk space, Python 3, Git, C++ Build Tools
2. TORCS + SCR	Download and unzip — no installer, just extract and run
3. Python packages	Create a virtual environment and install libraries
4. gym_torcs	Set up the AI starter kit provided by organisers
5. VS Code	Install the code editor and Python extension
6. Ollama + Granite	Set up the local AI coding assistant (offline, no API key)
7. Continue.dev	Connect the AI assistant to VS Code

Prefer a simpler setup?

If you would rather skip manual installation, use the pre-built container image with Podman — see the companion TORCS Container Setup Guide. Everything in this guide is already included in that image.

1. Prerequisites

1.1 Hardware requirements

Requirement	Details
OS	Windows 10 or Windows 11 (64-bit)
CPU	Any modern 64-bit processor. TORCS is lightweight — designed for 800 MHz CPUs, so any current machine is more than sufficient.
RAM	4 GB minimum. 8 GB recommended when running TORCS and the Python agent simultaneously. 16 GB if you plan to train neural networks.
Graphics card	Any GPU with OpenGL 1.3 support — this covers virtually every card made in the last 20 years including integrated Intel/AMD graphics. A dedicated GPU is not required for TORCS. An NVIDIA GPU with CUDA is only needed if you want faster neural network training.
Disk space	15 GB free (see breakdown below)
Display	Any monitor — TORCS runs in a standard window

1.2 Disk space breakdown

Component	Space needed
TORCS (extracted zip)	~500 MB
Python 3	~100 MB
Visual C++ Build Tools	~4 GB
VS Code	~400 MB
Python virtual env + packages (PyTorch, TensorFlow, etc.)	~5-6 GB
Ollama + Granite 4.0 350M model	~500 MB
Working space during installation	~2 GB
Total recommended free space	15 GB

No dedicated GPU needed for this competition

TORCS renders 3D graphics but its requirements are very modest — integrated graphics on any laptop from the last 10 years will run it fine. A dedicated GPU only matters if you want to speed up neural network training, which is optional.

1.3 Software prerequisites

Install these three tools before starting. All use standard Windows installers.

1.4 Python 3

1. Go to <https://www.python.org/downloads/> and download the latest Python 3 installer.
2. Run the installer. On the first screen tick 'Add Python to PATH' before clicking Install Now — this is critical.
3. Open PowerShell (press Windows key, type PowerShell, press Enter) and verify:

```
python --version
```

Expected output: Python 3.12.x or similar.

1.5 Git for Windows

1. Go to <https://git-scm.com/download/win> and download the installer.
2. Run with all default settings.
3. Verify in PowerShell:

```
git --version
```

1.6 Visual C++ Build Tools

Required for compiling some Python packages. Download from:

```
https://visualstudio.microsoft.com/visual-cpp-build-tools/
```

1. Run the installer, select the 'Desktop development with C++' workload, and click Install.

2. TORCS + SCR Patch

TORCS on Windows runs directly from a folder — no formal installation needed. The SCR patch (the UDP socket interface your AI connects to) and the competition car designs are all included in the zip file provided by the organisers.

2.1 Download

1. Download the TORCS competition package from the link provided by your organiser:

```
https://ibm.ent.box.com/v/TORCSdownloadzip
```

2.2 Extract

1. Right-click the downloaded zip and select Extract All.
2. Extract to a simple path with no spaces, for example:

```
C:\torcs\
```

3. After extraction the folder should contain:

```
C:\torcs\torcs.exe           <- the simulator
C:\torcs\drivers\scr_server\0\ <- car designs (slots 0-9)
C:\torcs\tracks\            <- race tracks
```

2.3 Test TORCS

1. Open C:\torcs\ in File Explorer and double-click torcs.exe.
2. The TORCS main menu should appear. If Windows shows a security warning click 'More info' then 'Run anyway'.
3. Close TORCS for now — you will use it again in section 4.

Add TORCS to PATH (optional)

To be able to type 'torcs' from any PowerShell window, open PowerShell as Administrator and run: `[System.Environment]::SetEnvironmentVariable('Path', $env:Path + ';C:\torcs', 'User')` — then restart PowerShell.

3. Python Environment

A virtual environment keeps competition packages isolated from the rest of your system.

3.1 Create and activate

```
python -m venv C:\torcs-env  
C:\torcs-env\Scripts\activate
```

Your prompt will show (torcs-env) at the start confirming it is active. You need to run the activate command each time you open a new PowerShell window.

3.2 Install packages

With the virtual environment activated:

```
pip install numpy scipy scikit-learn torch tensorflow keras gymnasium  
matplotlib pandas
```

GPU / CUDA support

If you have an NVIDIA GPU, visit <https://pytorch.org/get-started/locally/> and use the custom PyTorch install command for your CUDA version for faster AI training.

4. gym_torcs Starter Kit

The gym_torcs folder is provided by the competition organisers:

<https://ibm.box.com/s/357vjo7tjrx142nwa5rli0p2kiukmmyo> (in the files folder).

It contains the Python wrapper that connects your AI code to TORCS and the starter agent you will modify.

4.1 Set up

1. Create your competition folder:

```
mkdir C:\RaceYourCode
```

2. Copy the gym_torcs folder from the organisers into it so the structure is:

```
C:\RaceYourCode\gym_torcs\torcs_jm_par.py
C:\RaceYourCode\gym_torcs\snakeoil3_gym.py
C:\RaceYourCode\gym_torcs\gym_torcs.py
C:\RaceYourCode\gym_torcs\practice.xml
```

4.2 Key files

File	Purpose
torcs_jm_par.py	Main starter file — this is what you edit for the competition
snakeoil3_gym.py	UDP client — connects Python to TORCS on port 3001. Do not edit.
gym_torcs.py	OpenAI Gym-style RL environment wrapper for advanced users
practice.xml	Race configuration file

4.3 Test the full connection

1. Start TORCS (double-click C:\torcs\torcs.exe).
2. In TORCS: Race → Practice → Configure Race → select a track → set scr_server as driver → Accept.
3. Race → Practice → New Race. TORCS shows a blue waiting screen — this is correct.
4. Open PowerShell, activate the virtual environment, and run:

```
C:\torcs-env\Scripts\activate
cd C:\RaceYourCode\gym_torcs
```

```
python torcs_jm_par.py
```

5. You should see 'Client connected on 3001' and the car starts moving on track.

Congratulations!

If the car is moving and the terminal shows 'Client connected', the full stack is working. You are ready to start modifying `torcs_jm_par.py`.

5. VS Code

5.1 Install

1. Download from <https://code.visualstudio.com/> and run the installer with default settings.

5.2 Install the Python extension

1. Press Ctrl+Shift+X to open Extensions.
2. Search for Python and install the Microsoft Python extension.

5.3 Set the Python interpreter

1. Press Ctrl+Shift+P and type: Python: Select Interpreter
2. Select the torcs-env at C:\torcs-env\Scripts\python.exe

5.4 Open your competition folder

```
code C:\RaceYourCode\gym_torcs
```

Click on torcs_jm_par.py to open the main competition file.

6. Ollama + IBM Granite 4.0 350M

Ollama runs the Granite AI model locally on your machine. It provides AI code assistance inside VS Code with no internet connection required and no API key needed.

6.1 Install Ollama

1. Download the Windows installer from:

```
https://ollama.com/download/windows
```

2. Run the installer. Ollama installs as a Windows service and starts automatically — no manual launching needed.

3. Verify in PowerShell:

```
ollama --version
```

6.2 Pull the Granite model

```
ollama pull granite4:350m
```

Downloads approximately 250 MB. Verify when complete:

```
ollama list
```

You should see granite4:350m listed.

7. Continue.dev — AI Assistant in VS Code

7.1 Install the extension

1. In VS Code press Ctrl+Shift+X.
2. Search for Continue and install the Continue extension by Continue.dev.

7.2 Configure to use Granite

1. Open File Explorer and navigate to:

```
C:\Users\YourName\.continue\
```

(Replace YourName with your actual Windows username. Create the .continue folder if it does not exist.)

2. Create a file named config.json with this content:

```
{
  "models": [
    {
      "title": "Granite 4.0 350M (local)",
      "provider": "ollama",
      "model": "granite4:350m",
      "apiBase": "http://localhost:11434"
    }
  ],
  "tabAutocompleteModel": {
    "title": "Granite 4.0 350M (autocomplete)",
    "provider": "ollama",
    "model": "granite4:350m",
    "apiBase": "http://localhost:11434"
  },
  "allowAnonymousTelemetry": false
}
```

7.3 Verify

1. Press Ctrl+Shift+P → Reload Window.
2. Click the Continue icon in the left VS Code sidebar.
3. Ask something like: What does STEER_GAIN do in this file?
4. Granite should respond within a few seconds.

The AI assistant works offline

Granite 4.0 350M runs entirely inside the container. It does not send your code to the internet. This means it works even without an internet connection during the competition.

You can also configure Granite served by <https://replicate.com/>. Use this code to get free credit: <https://replicate.com/invites/a8717bfe-2f3d-4a52-88ed-1356231cdf03>
Find suitable Granite code model, get API and configure it in Continue.dev extension.
Credit should give you ~1M tokens free of charge.

9. Your First Modification

Now that everything is working, here is how to make your first change to the AI agent and see the effect on track.

9.1 Understanding the parameters

Open `torcs_jm_par.py` in VS Code. Scroll to the section labelled **USER CONFIGURABLE PARAMETERS** (around line 493). You will see:

```
TARGET_SPEED = 100      # Target speed in km/h
STEER_GAIN = 30         # Steering sensitivity
CENTERING_GAIN = 0.20   # How strongly car stays centred
BRAKE_THRESHOLD = 0.9   # When to brake in corners
GEAR_SPEEDS = [0, 20, 40, 80, 100, 180] # Gear shift points
ENABLE_TRACTION_CONTROL = True
```

9.2 Make a change

1. Try changing `TARGET_SPEED` from 100 to 150:

```
TARGET_SPEED = 150
```

2. Save the file with **Ctrl + S**.
3. Make sure TORCS is running and waiting (repeat section 6 if needed).
4. In the terminal, run the agent again:

```
python3 torcs_jm_par.py
```

5. Watch the car on track. It should now drive faster. Notice whether it stays on track or spins out — you may need to also adjust `STEER_GAIN` or `BRAKE_THRESHOLD` to compensate.

Tip: iterate quickly

The typical competition workflow is: change a parameter, save, run the agent, watch the car, stop the agent (**Ctrl+C**), change again. Each run only takes a few seconds to start. Do not be afraid to experiment.

9.3 What each parameter affects

Parameter	What to try
TARGET_SPEED	Increase for faster lap times, but the car may spin out in corners

STEER_GAIN	Increase if the car is drifting sideways, decrease if it oversteers
CENTERING_GAIN	Increase if the car wanders; decrease if it makes too sharp corrections
BRAKE_THRESHOLD	Decrease (e.g. 0.5) to brake earlier before corners
GEAR_SPEEDS	Adjust shift points for more aggressive or conservative gear changes
ENABLE_TRACTION_CONTROL	Set to False to see what happens without it (spoiler: wheelspin)

10. Troubleshooting

Here are solutions to the most common problems students encounter.

Python script says 'Connection refused' or keeps waiting

This means the Python agent cannot find TORCS. Check the following:

- TORCS must be running and showing the blue waiting screen before you run the Python script
- Make sure you started a New Race in Practice mode with scr_server as the driver
- Run both TORCS and the Python script inside the same container (in the browser desktop), not on your own machine

The car immediately goes off track and stops

This is normal behaviour with the default parameters on some tracks. The example agent is tuned for moderate-difficulty tracks. Try:

- Reducing TARGET_SPEED to 80 or 60
- Increasing CENTERING_GAIN to 0.4
- Choosing a simpler track in TORCS (a straight or gentle oval)

The AI assistant (Continue.dev) is not responding

The Granite model needs a moment to load when the container first starts. Wait 30 seconds after the desktop appears before using the AI assistant. If it still does not respond, open a terminal inside the container and check Ollama is running:

```
ollama list
```

You should see granite4:350m listed. If not, run:

```
ollama pull granite4:350m
```

11. Quick Reference Card

Key files

File	What it is
/home/student/workspace/gym_torcs/torcs_jm_par.py	Main starter file — edit this for the competition
/home/student/workspace/gym_torcs/snakeoil3_gym.py	Core UDP client — do not edit
/home/student/workspace/gym_torcs/gym_torcs.py	RL environment wrapper — advanced users
/home/student/workspace/gym_torcs/practice.xml	Race configuration

Key paths

Item	Path
TORCS executable	C:\torcs\torcs.exe
SCR car designs	C:\torcs\drivers\scr_server\
Virtual environment	C:\torcs-env\
Competition code	C:\RaceYourCode\gym_torcs\
Continue.dev config	C:\Users\YourName\continue\config.json
Ollama models	%USERPROFILE%\ollama\models\

Key commands (PowerShell)

Task	Command
Activate virtual env	C:\torcs-env\Scripts\activate
Run example agent	python torcs_jm_par.py
List Ollama models	ollama list
Open workspace in VS Code	code C:\RaceYourCode\gym_torcs
Stop the agent	Ctrl + C

Typical session

1. Double-click `C:\torcs\torcs.exe` to start TORCS
2. In TORCS: Race → Practice → New Race (`scr_server` must be the configured driver)
3. Open PowerShell and activate your virtual environment
4. `cd C:\RaceYourCode\gym_torcs`
5. `python torcs_jm_par.py`
6. Watch the car, stop with Ctrl+C, edit your code, repeat

Good luck and happy racing!